

React Native with TypeScript:

Intermediate Tutorial

This picks up where the beginner tutorial left off. We'll add TypeScript for type safety and cover patterns you need for real apps: typed props, custom hooks, context, typed navigation, and API data fetching.

Assumes you're comfortable with components, `useState`, and basic navigation.

1. Setting Up a TypeScript Project

```
npx create-expo-app@latest MyApp --template
```

Choose the **TypeScript** template when prompted. Or convert an existing JS project: rename `.js` files to `.tsx` (for files with JSX) or `.ts` (plain logic), then add:

```
npx expo install typescript @types/react
@types/react-native
```

You'll get a `tsconfig.json` — Expo's default config is fine to start with.

2. Typing Props

Every component's props should have an explicit shape. Use an `interface` or `type`:

```
import { View, Text, StyleSheet } from 'react-native';

interface ProfileCardProps {
  name: string;
  age: number;
  isOnline?: boolean; // optional prop
  onPress: () => void;
}

function ProfileCard({ name, age, isOnline = false, onPress }: ProfileCardProps) {
  return (
    <View style={styles.card}>
      <Text style={styles.name}>{name}, {age}</Text>
      <Text style={{ color: isOnline ? 'green' : 'gray' }}>
        {isOnline ? 'Online' : 'Offline'}
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  card: { padding: 16, borderRadius: 8,
    backgroundColor: '#f2f2f2' },
  name: { fontSize: 18, fontWeight: '600' },
});

export default ProfileCard;
```

Why this matters: if a parent forgets to pass `name`, or passes a number instead of a string, TypeScript catches it at compile time — before it ever reaches your phone.

3. Typing State and Events

```
import { useState } from 'react';
import { View, TextInput, StyleSheet } from
'react-native';

interface User {
  id: string;
  name: string;
  email: string;
}

export default function App() {
  // Primitive state: TypeScript infers the type
  automatically
  const [count, setCount] = useState(0);

  // Complex/nullable state: be explicit
  const [user, setUser] = useState<User | null>
(null);

  // Array state
  const [tags, setTags] = useState<string[]>
([]);

  const handleChangeText = (text: string) => {
    setTags(text.split(','));
  };

  return (
    <View style={styles.container}>
      <TextInput onChangeText={handleChangeText}
```

```
placeholder="tag1, tag2" />
  </View>
);
}

const styles = StyleSheet.create({ container: {
flex: 1, padding: 20 } });
```

`useState<User | null>(null)` is a common pattern: state starts as `null` (nothing loaded yet) and later becomes a `User` object once you have data.

4. Typing FlatList

Generic components like `FlatList` need a type parameter so `item` is correctly typed inside `renderItem`:

```
import { FlatList, Text, View } from 'react-native';

interface Todo {
  id: string;
  title: string;
  done: boolean;
}

const todos: Todo[] = [
  { id: '1', title: 'Learn TypeScript', done: false },
  { id: '2', title: 'Build an app', done: false },
];

export default function TodoList() {
  return (
```

```

<FlatList<Todo>
  data={todos}
  keyExtractor={({item}) => item.id}
  renderItem={({ item }) => (
    <View style={{ padding: 12 }}>
      <Text>{item.title} – {item.done ? '✅'
: '⌚'}</Text>
    </View>
  )}
/>
);
}

```

Now `item.title` autocompletes, and referencing `item.titel` (typo) is a compile error, not a silent undefined.

5. Custom Hooks with Types

Custom hooks are where TypeScript really pays off — they define a clear contract for what a piece of logic returns.

```

import { useState, useEffect } from 'react';

interface Post {
  id: number;
  title: string;
  body: string;
}

interface UsePostsResult {
  posts: Post[];
  loading: boolean;
  error: string | null;
}

```

```

function usePosts(): UsePostsResult {
  const [posts, setPosts] = useState<Post[]>
  ([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string |
  null>(null);

  useEffect(() => {
    let isMounted = true;

    async function fetchPosts() {
      try {
        const response = await
fetch('https://jsonplaceholder.typicode.com/post
s?_limit=10');
        if (!response.ok) throw new
Error('Failed to fetch posts');
        const data: Post[] = await
response.json();
        if (isMounted) setPosts(data);
      } catch (err) {
        if (isMounted) setError(err instanceof
Error ? err.message : 'Unknown error');
      } finally {
        if (isMounted) setLoading(false);
      }
    }

    fetchPosts();
    return () => { isMounted = false; };
  }, []);

  return { posts, loading, error };
}

export default usePosts;

```

Using it in a component:

```
import { View, Text, FlatList, ActivityIndicator } from 'react-native';
import usePosts from './usePosts';

export default function PostsScreen() {
  const { posts, loading, error } = usePosts();

  if (loading) return <ActivityIndicator style={{ flex: 1 }} />;
  if (error) return <Text>Error: {error}</Text>;

  return (
    <FlatList
      data={posts}
      keyExtractor={({item}) =>
        item.id.toString()}
      renderItem={({ item }) => (
        <View style={{ padding: 16,
          borderBottomWidth: 1, borderColor: '#eee' }}>
          <Text style={{ fontWeight: '600' }}>
            {item.title}</Text>
          </View>
        </View>
      )}
    />
  );
}
```

The `isMounted` flag prevents a memory-leak warning if the component unmounts before the fetch resolves — a common gotcha in real apps.

6. Typed Context (Global State)

Context is useful for things like auth state or theme, shared across many screens.

```
import { createContext, useContext, useState,
ReactNode } from 'react';

interface AuthContextValue {
  user: string | null;
  login: (username: string) => void;
  logout: () => void;
}

const AuthContext =
createContext<AuthContextValue | undefined>
(undefined);

export function AuthProvider({ children }: {
children: ReactNode }) {
  const [user, setUser] = useState<string |
null>(null);

  const login = (username: string) =>
setUser(username);
  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login,
logout }}>
      {children}
    </AuthContext.Provider>
  );
}

// Custom hook to consume the context safely
export function useAuth(): AuthContextValue {
  const context = useContext(AuthContext);
  if (!context) {
```

```
        throw new Error('useAuth must be used within  
an AuthProvider');  
    }  
    return context;  
}
```

Wrap your app once:

```
export default function App() {  
    return (  
        <AuthProvider>  
            <MainNavigator />  
        </AuthProvider>  
    );  
}
```

Then anywhere inside:

```
function ProfileScreen() {  
    const { user, logout } = useAuth();  
    return <Text>Welcome, {user}</Text>;  
}
```

The `undefined` default plus the runtime check in `useAuth` guarantees you never accidentally use the context outside its provider — TypeScript and a runtime guard working together.

7. Typed Navigation

React Navigation's real strength with TypeScript is catching invalid routes or missing params.

```
npx expo install @react-navigation/native
@react-navigation/native-stack
```

```
import { NavigationContainer } from '@react-
navigation/native';
import { createNativeStackNavigator,
NativeStackScreenProps } from '@react-
navigation/native-stack';
import { View, Text, Button } from 'react-
native';

// 1. Define every screen and the params it
expects
type RootStackParamList = {
  Home: undefined;           // no params
  Details: { itemId: string }; // requires
itemId
};

const Stack =
createNativeStackNavigator<RootStackParamList>
();

// 2. Type each screen's props using the param
list
type HomeProps =
NativeStackScreenProps<RootStackParamList,
'Home'>;
type DetailsProps =
NativeStackScreenProps<RootStackParamList,
'Details'>;

function HomeScreen({ navigation }: HomeProps) {
  return (
    <View style={{ flex: 1, alignItems:
'center', justifyContent: 'center' }}>
```

```

        <Button
          title="View Item 42"
          onPress={() =>
            navigation.navigate('Details', { itemId: '42'
          })}
        />
      </View>
    );
  }

  function DetailsScreen({ route }: DetailsProps)
  {
    const { itemId } = route.params; // fully
    typed, no casting needed
    return (
      <View style={{ flex: 1, alignItems:
        'center', justifyContent: 'center' }}>
        <Text>Item ID: {itemId}</Text>
      </View>
    );
  }

  export default function App() {
    return (
      <NavigationContainer>
        <Stack.Navigator>
          <Stack.Screen name="Home" component=
            {HomeScreen} />
          <Stack.Screen name="Details" component=
            {DetailsScreen} />
        </Stack.Navigator>
      </NavigationContainer>
    );
  }

```

Now `navigation.navigate('Detials', ...)` (typo) or
`navigation.navigate('Details')` (missing required

`itemId`) both fail at compile time.

8. Bringing It Together: A Typed API-Backed List

A small but realistic screen: fetch data, type it, handle loading/error states, and navigate to a detail view.

```
interface Product {
  id: number;
  title: string;
  price: number;
}

type ProductStackParamList = {
  ProductList: undefined;
  ProductDetail: { product: Product };
};

const Stack =
  createNativeStackNavigator<ProductStackParamList>();

function ProductListScreen({
  navigation,
}: NativeStackScreenProps<ProductStackParamList,
'ProductList'> ) {
  const [products, setProducts] =
    useState<Product[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('https://fakestoreapi.com/products')
      .then((res) => res.json())
      .then((data: Product[]) =>
```

```

setProducts(data))
    .finally(() => setLoading(false));
}, []);

if (loading) return <ActivityIndicator style=
{{ flex: 1 }} />;

return (
  <FlatList<Product>
    data={products}
    keyExtractor={({item) =>
item.id.toString()}
    renderItem={({ item }) => (
      <Pressable
        style={{ padding: 16,
borderBottomWidth: 1, borderColor: '#eee' }}
        onPress={() =>
navigation.navigate('ProductDetail', { product:
item })}
      >
        <Text>{item.title} - ${item.price}
</Text>
      </Pressable>
    )}
  />
);
}

function ProductDetailScreen({
  route,
}: NativeStackScreenProps<ProductStackParamList,
'ProductDetail'>) {
  const { product } = route.params;
  return (
    <View style={{ flex: 1, padding: 20 }}>
      <Text style={{ fontSize: 22, fontWeight:
'bold' }}>{product.title}</Text>

```

```
    <Text style={{ fontSize: 18, marginTop: 8
    }}>${product.price}</Text>
  </View>
);
}
```

Passing the whole `product` object as a route param avoids a second fetch on the detail screen — and TypeScript ensures both screens agree on its shape.

Common Pitfalls

- **Avoid `any`**. It silences TypeScript entirely. If you don't know a type yet, use `unknown` and narrow it, or define a proper interface.
- **Style objects benefit from `StyleSheet` typing**. If you build styles dynamically, type them as `ViewStyle`, `TextStyle`, or `ImageStyle` from `react-native`.
- **`useRef` needs a type argument** for DOM-like refs: `useRef<TextInput>(null)`.
- **Don't over-type simple `useState` calls** — `useState(0)` infers `number` fine on its own; only add `<T>` when the initial value doesn't reveal the full type (like `null` placeholders).

Next Steps

- Explore **Zod** or **io-ts** to validate API responses at runtime — TypeScript types disappear at runtime, so validation catches malformed server data.

- Look into **React Query (TanStack Query)** with TypeScript for robust data fetching, caching, and typed query results.
- Try **strict mode** in `tsconfig.json` (`"strict": true`) once comfortable — it catches far more issues, like implicit `any` and unchecked `null`s.